

Lab 1: Interactive Image Mosaic Generator

Pinar Aksoy CS5130, Fall 2026 Code: <https://github.com/0xpinara/mosaic-hf>

Live demo: <https://huggingface.co/spaces/piax0x/mosaic-hf>

1 Method

Preprocessing. All the image processing code is in `mosaic.py`. The grid size N is the number of tiles along the long side of the image, and each cell is $\lfloor 1024/N \rfloor$ pixels wide. I resize the image so that its long side is exactly N cells. So for a given N , every image gets the same fixed long side of at most 1024 pixels. Then I crop the short side from the center so that it also fits a whole number of cells, which cuts off less than one cell.

Grid analysis and classification. To get the average color of every cell without a loop, I reshape the $H \times W \times 3$ image into a $rows \times cell \times cols \times cell \times 3$ array. The pixels of each cell are then on axes 1 and 3, so one mean over these two axes gives all the cell colors at once.

Next, k-means color quantization with `cv2.kmeans` groups the cell colors into k categories. NumPy then finds the distance from every cell to every category in one step, and `argmin` labels each cell with the closest category. So the category borders act as color thresholds. Both steps use CIELAB instead of RGB, because CIELAB distances are closer to how different colors look. Figure 1b shows the segmented image, with each cell in its category color.

Building the mosaic. Each tile pattern is drawn once at 128×128 pixels, scaled down to the cell size, and then colored with all k category colors at once. The mosaic is simply `tiles[labels]`, which picks the right tile for every cell. A transpose and a reshape turn it back into one image, so this step has no loop over the cells either.

Tile sets. There are five tile sets, shown in Figure 1. *LEGO bricks* shade the color like a small brick seen from above. *Flat colors* are plain squares, and they are the baseline. *Dots* are round beads on a dark background, like an LED board. *ASCII* uses intensity thresholding: the brightness range is split into 11 levels, one for each character. Brighter cells get characters with more ink, drawn in color on a near-black background. *Photo tiles* do not use k-means. I cut the seven test photos into 592 patches of 64×64 pixels, and each cell gets the patch with the closest average color. Then the patch color is moved 40% toward the cell color. If the input is one of the test photos, its own patches are left out. Without this rule, the astronaut with an 8×8 grid came back almost unchanged, with an SSIM of 0.999.

Interface. The Gradio app is in `app.py`. The user uploads an image, sets the grid size from 8 to 128 and k with two sliders, and picks the tile set from a menu. It then shows the resized original, the segmented image, the mosaic, the two metrics and the processing time. The output updates every time an input changes. Eight example buttons cover all seven test images and all five tile sets, with the settings of Figure 2. The app runs on Hugging Face Spaces.

2 Similarity metrics

I compare each mosaic with the resized original. MSE is the mean squared error of the 0 to 255 RGB values, so lower is better. SSIM, from `scikit-image`, compares local brightness, contrast and structure instead of single pixels. It is 1 for identical images, so higher is better. For Table 1, I cropped the seven images to squares, so every grid is exactly $N \times N$.

Table 1: MSE and SSIM between the original and the mosaic for each grid size. Mean of the 7 images, $k = 16$.

Tile set	MSE, lower is better				SSIM, higher is better			
	16×16	32×32	64×64	128×128	16×16	32×32	64×64	128×128
Flat colors	990	661	443	287	0.618	0.625	0.637	0.678
LEGO bricks	1265	930	692	444	0.405	0.319	0.256	0.304
Dots	3561	3300	3098	2722	0.406	0.303	0.189	0.155
ASCII	7871	7626	7326	6996	0.198	0.183	0.159	0.136
Photo tiles	3150	2578	2201	1763	0.284	0.236	0.185	0.162

For every tile set, MSE goes down as the grid gets finer, because smaller cells follow the image more closely. Flat colors always score best, since any texture inside a tile counts as error.

SSIM behaves differently. For the four textured sets, it goes down as the grid gets finer, even though MSE gets better. With small cells, the texture inside the tiles becomes the main local structure, and this texture does not match the photo. LEGO goes up a little at 128×128 , likely because the tiny 8-pixel bricks have softer shading. Photo tiles are third in MSE but near the bottom in SSIM. Their colors match each cell only on average, and their edges rarely line up with the edges in the image. ASCII comes last, because most of each tile is near-black.

The image matters too. At 32×32 with Flat colors, the smooth rocket image has an MSE of 221, but the astronaut has 1360, because small details like its flag and badges are smaller than one cell. So the metrics measure how close a



Figure 1: The astronaut image with a 32×32 grid and $k = 16$. Panels a to h show the resized original, the segmented image, the five tile sets and a close-up of the Photo tiles, which use patches from the other six photos.

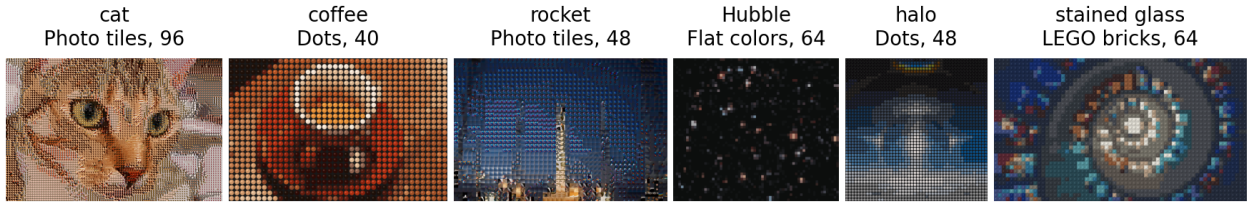


Figure 2: The other six test images, with $k = 16$. The tile set and grid size are shown above each one.

mosaic is to the original, not how good it looks. To me, LEGO and Photo tiles look better than their scores suggest.

3 Performance

I timed the grid steps with `benchmark.py`: finding cell colors, classifying cells and placing tiles. A second version does the same steps with two nested `for` loops over the cells. It gives exactly the same mosaic, as `np.array_equal` confirms. Each value is the median of repeated runs on my Apple M2 Mac with Python 3.12, NumPy 2.4.4 and OpenCV 4.13.

Table 2: Processing time in ms for each grid size. Average of the 7 images at 1024×1024 pixels, with LEGO bricks and $k = 16$.

Grid	Cells	Cell size	Vectorized	Nested loops	Speedup	Full pipeline	Metrics
8×8	64	128 px	10.8	11.4	$1.1\times$	23.3	249.3
16×16	256	64 px	10.9	12.0	$1.1\times$	20.2	228.7
32×32	1024	32 px	10.8	17.4	$1.6\times$	19.6	203.0
64×64	4096	16 px	12.7	41.6	$3.3\times$	29.3	212.8
128×128	16384	8 px	19.6	140.7	$7.2\times$	37.7	213.3

The vectorized time barely changes with the grid size. From 8×8 to 128×128 , the number of cells grows 256 times, but the time only goes from 10.8 to 19.6 ms. This is because most of the work is per pixel, and the pixel count stays the same. The loop version does the same 11 ms of pixel work, but it also spends about 8 μ s in Python on every cell.

With few cells, this extra time does not matter. With many cells, it takes most of the time: from 64×64 to 128×128 , there are 4 times more cells, and the loop takes 3.4 times longer. At 128×128 , the vectorized code is 7.2 times faster. The whole pipeline takes only 20 to 38 ms, so computing the metrics, at about 0.2 s, is the slowest step.

4 Conclusion

The generator works on images of any size and shape and is fast enough to feel interactive. The metrics show a trade-off between accuracy and style: the more interesting tile sets lose points for the texture that makes them look like mosaics. Next, I would add a bigger photo library, because now the same patches repeat a lot and need a strong color shift.